

Projet Mineure SDCI — État d'avancement par phase

Contexte général

Le projet vise à simuler un réseau complet de capteurs, une gateway intermédiaire et un serveur central. L'objectif est de faire circuler les données des capteurs jusqu'au serveur, tout en surveillant et adaptant le comportement de la gateway en cas de saturation.

Phase 1 : simulation réseau

Objectif

- Simuler l'architecture Serveur → GI → Zones
- Relier des capteurs via des gateways et faire arriver les données jusqu'au serveur
- Vérifier que le réseau se monte correctement et que les paquets peuvent transiter

Réalisé

- `reseau.py` construit une topologie Mininet/EmuVIM
- Une datacenter `dc1` est créé avec les APIs OpenStack/REST
- Les conteneurs Docker sont ajoutés :
 - `Serveur` (IP 10.0.0.5)
 - `GI` (IP 10.0.0.10)
 - `Zone1` (IP 10.0.0.20)
 - `Zone2` (IP 10.0.0.30)
 - `Zone3` (IP 10.0.0.40)
- Les liens et switches sont connectés afin de créer le réseau
- Les entypoints des conteneurs démarrent les gateways et le device
- Le flux de paquets depuis les zones vers la GI et le serveur fonctionne correctement

Statut

- Phase 1 : **complète**
 - Le réseau est fonctionnel : les conteneurs démarrent, les liens sont établis et les paquets circulent jusqu'au serveur
-

Phase 2 : monitoring et adaptation

Objectif

- Surveiller l'état de la gateway intermédiaire (GI)
- Détecter la saturation
- Appliquer des actions d'adaptation lorsque la GI est surchargée
- Ajouter un second scénario d'optimisation des données en cas de congestion

Ce qui existe déjà

- `moniteur.py` : script de surveillance
 - interroge `http://172.17.0.3:8181/health`
 - lit le champ `currentLoad`
 - compare à un seuil de 20 %
 - en cas de saturation, déclenche `action()`
- `action()` effectue un PUT vers l'API REST OpenStack :
 - `http://127.0.0.1:5001/restapi/compute/dcl/gi_secours`
 - `payload: { "image": "gi_vnf", "network": "(id=vnf-eth0)" }`
- Le script affiche les logs de charge et passe en action quand le seuil est dépassé

À compléter

- Implémentation du second scénario d'adaptation :
 - agrégation des données du capteur saturant le réseau
 - lissage des données avant envoi
 - envoi de requêtes moins saturantes au serveur central
 - L'optimisation des flux de données n'est pas encore mise en œuvre
-

Ajouts récents réalisés dans le projet

Dropper aléatoire

- Ajout d'un gateway local dans `dockerfiles/gateway.js`
 - implémente un paramètre `--drop_rate`
 - ajoute la fonction `shouldDrop()`
 - sur POST `/device/:dev/data`, le message peut être abandonné aléatoirement
 - si le message est droppé, la gateway renvoie une réponse 200 OK avec `{ dropped: true, message: 'Dropped by gateway' }`

Mise à jour des Dockerfiles

- `dockerfiles/GI.Dockerfile`
 - remplace le téléchargement distant de `gateway.js` par un COPY `gateway.js`
`/gateway/gateway.js`
- `dockerfiles/Zone.Dockerfile`
 - remplace également le téléchargement distant de `gateway.js` par un COPY `gateway.js`
`/gateway/gateway.js`

Mise à jour des entrypoints

- `dockerfiles/gateway.sh`
- `dockerfiles/zone.sh`

Ces scripts démarrent maintenant la gateway avec :

```
--drop_rate "${DROP_RATE:-0.1}"
```

- Par défaut, le taux de drop est de 0.1 (10 %)
- Il peut être ajusté via la variable d'environnement `DROP_RATE`

Fichiers importants du projet

- `reseau.py` : topologie réseau et démarrage Mininet/EmuVIM
- `moniteur.py` : surveillance de la GI et adaptation par scale-out
- `dockerfiles/GI.Dockerfile` : image de la GI avec gateway
- `dockerfiles/Zone.Dockerfile` : image de Zone avec gateway + device
- `dockerfiles/gateway.sh` : script d'entrée de la GI
- `dockerfiles/zone.sh` : script d'entrée des zones
- `dockerfiles/gateway.js` : gateway locale avec dropper aléatoire

Prochaines étapes

1. Implémenter l'agrégation et le lissage des données du capteur saturant
2. Ajouter la logique d'envoi de requêtes allégées au serveur central
3. Tester le système complet avec le dropper activé
4. Valider que l'optimisation des flux réduit effectivement la charge sur le réseau